Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного бюджетного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ МАШИНОСТРОИТЕЛЬНЫЙ

КАФЕДРА МЕХАТРОНИКА И РОБОТЕТХНИЧЕСКИЕ СИСТЕМЫ

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:

«Программирование управления мехатронной системой»

по дисциплине «Алгоритмическое и программное обеспечение мехатронных и робототехнических систем»

Студент гр. МК7-31М _____(Гайгеров М.А.)
(подпись) (Ф.И.О.)

Руководитель _____(Пащенко В.Н.)
(подпись) (Ф.И.О.)

Оценка руководителя_____баллов _____
30-50 (дата)

Оценка защиты _____баллов _____
30-50 (дата)

Оценка практики _____баллов _____
(оценка по пятибалльной шкале)

Комиссия:_____(_____)
(подпись) (Ф.И.О.)

_____(_____)
(подпись) (Ф.И.О.)

_____(_____)
(подпись) (Ф.И.О.)

Калуга, 2024

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного бюджетного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой **МК7**
_____(_Пащенко В.Н._)
«_____»_____20__г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине ***Алгоритмическое и программное обеспечение МХТ И РТС***

Студент группы ***МК7-31М Гайгеров Максим Алексеевич***
(фамилия, имя, отчество)

Тема курсового проекта ***Программирование управления мехатронной системой***

Направленность КП ***учебный***
Источник тематики ***кафедра МК7***

Задание

Описать функциональную схему, назначение, требования к точности и математическую модель мехатронной системы.
Описать цель управления, уровни и структурную схему системы управления.
Обосновать выбор компонентов электромеханической системы и принципиальную схему системы управления.
Реализовать выбранную систему управления и создать графическое приложение, описать обмен данными между мехатронной системой и программой.

Оформление курсового проекта
Расчетно-пояснительная записка на_____ листах формата А4.

Дата выдачи задания «___»_____ 2024 г.

Руководитель	_____ (подпись, дата)	Пащенко В.Н. (Ф.И.О.)
Студент	_____ (подпись, дата)	Гайгеров М.А. (Ф.И.О.)

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного бюджетного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсового проекта

по дисциплине ***Алгоритмическое и программное обеспечение МХТ И РТС***

Студент группы ***МК7-31М Гайгеров Максим Алексеевич***
(фамилия, имя, отчество)

Тема курсового проекта ***Программирование управления мехатронной системой***

№	Наименование этапов	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Руководитель	Куратор
1	Получение задания на выполнение курсового проекта	1-я нед.			
2	Разработка схемы и модели механизма	5-я нед.			
3	Описание системы управления	10-я нед.			
4	Создание графического интерфейса приложения	15-я нед.			
4	Защита курсового проекта	17-я нед.			

Студент _____ Руководитель _____
(подпись, дата) (подпись, дата)

Оглавление

Введение 5
1. Описание схемы и модели механизма 7
1.1 Функциональная схема мехатронной системы 7
2.1 Назначение 8
2.2 Математическая модель мехатронной системы 9
2.3.1 Прямая задача о положениях ..... **Error! Bookmark not defined.**
2.3.2 Обратная задача о положениях ..... **Error! Bookmark not defined.**
3. Анализ 3D-модели 11
4. Подбор электронных компонентов 15
5. Структурная схема системы управления 24
6. Создание графического приложения 29
Список использованных источников 37
ПРИЛОЖЕНИЕ А 39

Введение

В настоящее время управление манипуляционными механизмами на основе микропроцессорных систем представляет собой важное направление исследований в области робототехники и автоматизации. Это связано с возрастающим спросом на эффективные и гибкие решения для выполнения разнообразных задач в промышленности, медицине, аэрокосмической и других сферах[4].

Микропроцессорные системы играют ключевую роль в управлении манипуляторами, обеспечивая высокую точность, скорость и гибкость в выполнении операций. Они позволяют реализовывать различные алгоритмы управления, включая обратную связь, планирование траектории, адаптивное управление и многое другое. Благодаря быстрому развитию и миниатюризации микропроцессорных устройств, современные манипуляторы становятся более компактными, энергоэффективными и универсальными.

Одним из основных преимуществ использования микропроцессорных систем в управлении манипуляторами является возможность интеграции с другими компонентами системы, такими как датчики, актуаторы и интерфейсы пользователей[1]. Это позволяет создавать комплексные автоматизированные системы, способные адаптироваться к различным условиям и требованиям задачи.

Однако существуют и вызовы при разработке и применении микропроцессорных систем в управлении манипуляторами. Среди них - необходимость обеспечения высокой стабильности и надежности работы системы, учет особенностей динамики манипулятора при проектировании алгоритмов управления, а также обеспечение безопасности операций при работе с людьми и окружающей средой.

Целью данной курсовой работы является анализ основных аспектов управления манипуляционными механизмами на основе микропроцессорных систем с учетом современных технологических и научных достижений[2].

1. Описание схемы и модели механизма

1.1 Функциональная схема мехатронной системы

Мехатронная система, представленная в данном курсовом проекте, состоит из следующих основных компонентов (рисунок 1):

- Плата микроконтроллера: центральный элемент управления, отвечающий за обработку данных, выполнение алгоритмов управления и координацию работы других компонентов системы.
- Драйверы: три драйвера, каждый из которых предназначен для управления двумя двигателями.
- Двигатели: шесть двигателей, из которых четыре управляют рабочим органом, предназначенным для удержания рабочего инструмента, а два управляют поворотным столиком.
- Датчики тока: шесть датчиков, расположенных на соответствующих двигателях, предназначены для измерения тока, потребляемого каждым двигателем, что позволяет контролировать и оптимизировать процесс работы.
- Энкодеры: шесть абсолютных энкодеров, используемых для измерения угловых положений рабочего инструмента и поворотного столика с высокой точностью.
- Экран: используется для вывода информации о текущем текущем потреблении тока.

Управление мехатронной системой осуществляется с помощью графического приложения, разработанного на языке программирования Python. Пользовательский интерфейс приложения позволяет управлять различными параметрами работы системы, задавать требуемые угловые положения рабочего инструмента и поворотного столика, а также отслеживать текущее состояние и данные с датчиков [1].

2.1 Назначение

Назначение мехатронной системы заключается в автоматизации и оптимизации процессов, связанных с выполнением определенных задач, которые требуют управления механическими, электрическими и программными компонентами. В конкретном случае данной системы она предназначена для управления рабочим инструментом и поворотным столиком с целью выполнения различных операций с высокой точностью, надежностью и эффективностью [2].

Примеры назначения мехатронной системы могут включать в себя:

- Промышленное производство: мехатронные системы могут применяться в производственных линиях для автоматизации и оптимизации процессов сборки, обработки и упаковки продукции. В данном случае система может использоваться для манипулирования деталями, управления инструментами для обработки материалов, таких как фрезеровка, сверление или резка, а также для позиционирования и поворота изделий на конвейерах.
- Медицинская техника: в области медицинских технологий мехатронные системы могут применяться для выполнения хирургических операций с высокой точностью и контролем. Например, они могут использоваться в роботизированных системах для проведения минимально инвазивных операций, таких как лапароскопия или дистанционная хирургия.
- Лабораторные и научные исследования: мехатронные системы широко применяются в лабораториях и научных исследованиях для автоматизации экспериментов, манипулирования образцами и измерения различных параметров. Например, они могут использоваться в системах автоматической обработки и анализа биологических проб, в системах

тестирования материалов или в экспериментальных установках для изучения физических явлений.

- Обслуживание и ремонт: мехатронные системы также могут применяться для обслуживания и ремонта различных устройств и механизмов. Например, они могут использоваться в роботизированных системах для проведения диагностики, обслуживания и ремонта сложных оборудований, таких как автомобили, промышленные машины или аппараты для медицинских исследований.

В общем, назначение мехатронной системы определяется её специфическими характеристиками, требованиями заказчика и конкретными задачами, которые она предназначена решать. Однако, вне зависимости от конкретной области применения, основными целями мехатронных систем являются повышение производительности, улучшение качества выполняемых задач, снижение затрат и увеличение безопасности операций [3].

2.2 Математическая модель мехатронной системы

Пятизвенный механизм параллельной структуры с двигателями при основании является типом параллельного механизма, состоящего из пяти звеньев, которые соединены между собой и с основанием[2]. Двигатели расположены на основании и управляют движением каждого звена. Каждое звено может двигаться независимо от остальных, что позволяет осуществлять манипуляции с объектами в трехмерном пространстве. Этот тип механизма обычно используется в робототехнике и автоматизированных системах для выполнения различных задач, таких как сборка, позиционирование или перемещение предметов [7]. Кинематическая схема представлена на рисунке 1.

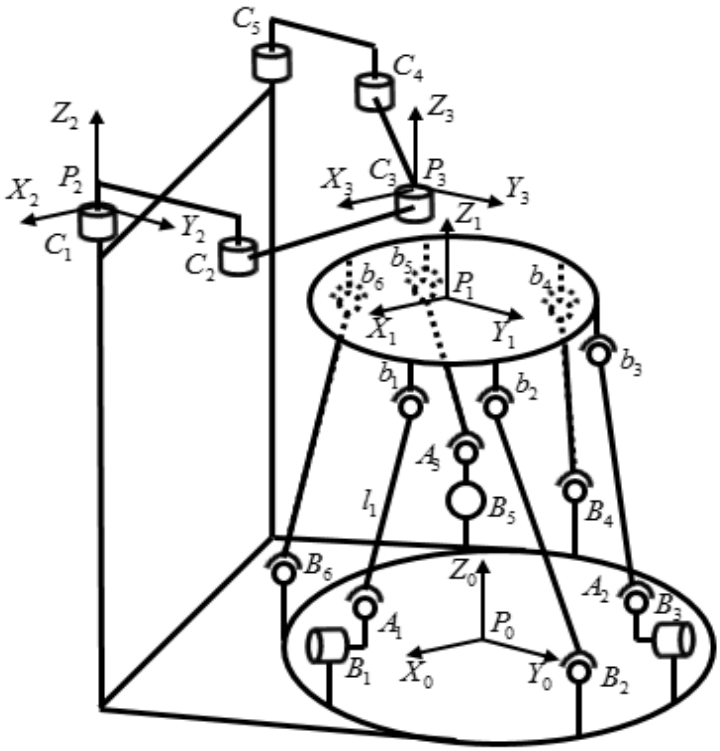

Рисунок 1 – Кинематическая схема механизма

$X_0Y_0Z_0$ - базовая неподвижная система координат;

$X_3Y_3Z_3$ - система координат поворотного столика;

$X_7Y_7Z_7$ - система координат пятизвенного механизма;

$\theta_3, \theta_4, \theta_5, \theta_6$ - обобщенные координаты плоского пяизвенного механизма;

θ_1, θ_2 - обобщенные координаты поворотного механизма.

Механизм совместного относительного манипулирования – пространственный механизм – включает основание, выходное звено модуля установки инструмента, выполненное в виде пятизвенного механизма, выходное звено модуля установки обрабатываемого объекта, выполненное в виде поворотного механизма[3]. Поворотный столик связан основанием установленной на основании вращательной кинематической парой, и еще одной вращательной кинематической парой, сопряженной непосредственно с поворотным столиком. Выходное звено пятизвенного механизма связано с установленными на основании двумя вращательными приводными

кинематическими парами и одной поступательной. Механизм выполнен в виде единой пространственной конструкции[6].

3. Анализ 3D-модели

Создание 3D – модели позволяет качественно проанализировать кинематические объекта исследования. Визуализация проектного стенда делает возможным дальнейший анализа конструктивных особенностей модели, характер кинематических взаимодействий между звеньями механизма [10].

В работе на уровне CAD – разработки применены графические пакеты компании SolidWorks. Инструменты системы обеспечивают полный цикл проектирования и создания конструкторской документации.

Исходя из конструктивных особенностей, в среде SolidWorks были спроектированы 3D – элементы всех деталей механизма и произведена общая сборка робота, представляющая собой детализированную модель.

Основу механизма параллельной структуры составляют 4 металлических звена и пластиковые сочленения, которые соединяют данные звенья. Помимо звеньев, на сочленения крепятся двигатели, обеспечивающие вращательное движение первых двух звеньев механизма, и энкодеры, обеспечивающие обратную связь об угле поворота и скорости движения звеньев[1].

Приводные и неприводные звенья механизма значительно различаются по конструкции.

Неприводные звенья (Рисунок 12) состоят из втулки, подшипников и трех корпусных деталей.

Приводные звенья (Рисунки 13-14) состоят из редуктора с передаточным отношением 1:4, двух корпусных деталей, двух подшипников и втулок.

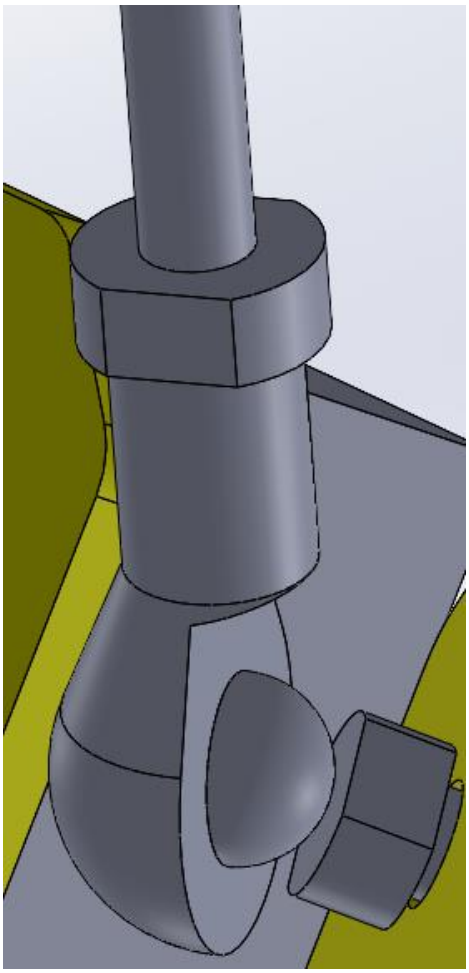

Рисунок 12 – Неприводное звено

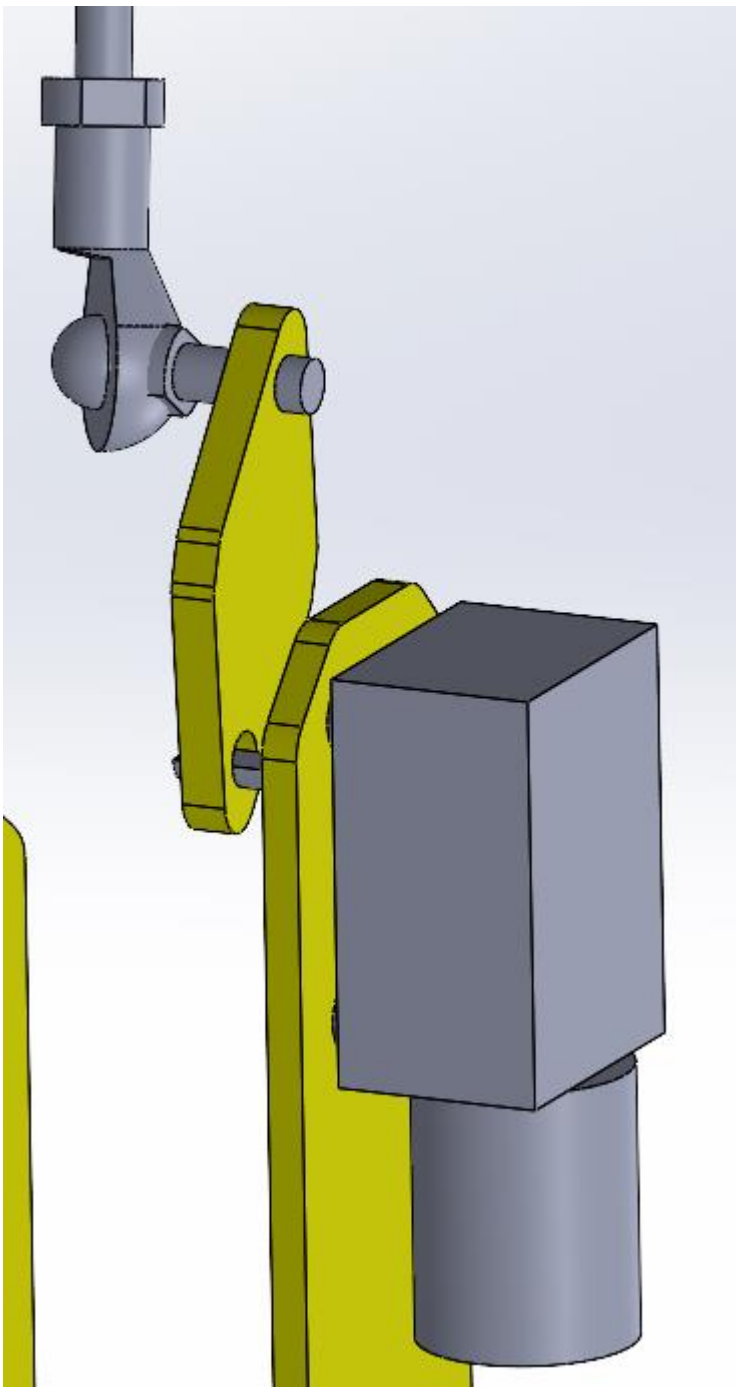

Рисунок 13 – Приводное сочленение

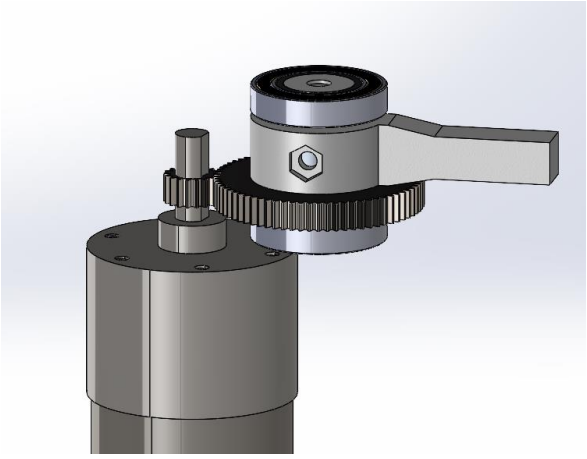

Рисунок 14 – Редуктор

Данная часть механизма крепится к металлической пластине с линейными подшипниками и кареткой передачи винт-гайка.

Плата Arduino управляется с помощью кода, написанного на компьютере. Датчики тока выводят текущие показания на микроконтроллер за тем на дисплей. Для управления моторами в качестве обратной связи используется датчик поворота[4].

Верхний уровень системы управления представлен компьютерной программой, входом которой является траектория, либо точка для рабочего органа.

Нижний уровень системы управления частично представлен в виде компьютерной программы, частично кодом, отвечающим за управление двигателями с помощью энкодеров на плате Arduino [8].

4. Подбор электронных компонентов

Использование Arduino Mega 2560 (16U2) (Рисунок 16) обосновано несколькими ключевыми преимуществами, которые делают эту платформу предпочтительной для определенных задач в области микроконтроллерного программирования и электроники [5].

Arduino Mega 2560 имеет большой объем памяти и более мощный микроконтроллер по сравнению с более стандартными моделями Arduino Uno.

Arduino Mega 2560 обладает большим количеством цифровых и аналоговых портов ввода/вывода, что делает его идеальным выбором для проектов, требующих управление большим числом датчиков, исполнительных устройств и других периферийных компонентов.

Наличие нескольких UART позволяет Arduino Mega 2560 легко обмениваться данными с несколькими внешними устройствами одновременно. Это особенно полезно в проектах, где требуется взаимодействие с различными периферийными устройствами[1].

Наличие двух процессоров – основного микроконтроллера и микроконтроллера для USB-интерфейса (16U2) – предоставляет расширенные возможности программирования.

Arduino Mega 2560 совместим с огромным количеством библиотек и расширений, что обеспечивает легкость интеграции различных функциональных возможностей в проектах. Это делает его универсальным выбором для широкого спектра приложений, от робототехники до умного дома.

Arduino Mega 2560 базируется на открытой аппаратной и программной инфраструктуре Arduino, что обеспечивает доступность и гибкость в разработке проектов.

С учетом вышеперечисленных преимуществ, Arduino Mega 2560 (16U2) становится предпочтительным выбором для различных проектов, требующих мощности, расширенных возможностей ввода/вывода и поддержки широкого спектра периферийных устройств [6] [7].

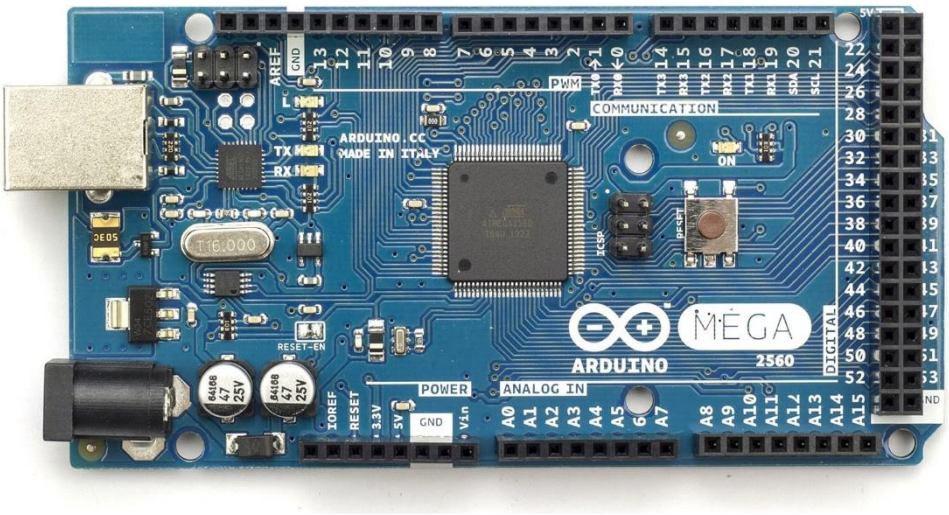

Рисунок 16 - Arduino Mega 2560 (16U2)

Технические характеристики:

- Микроконтроллер - ATmega2560
- Рабочее напряжение - 5В
- Входное напряжение (рекомендуемое) - 7-12В
- Входное напряжение (предельное) - 6-20В
- Цифровые Входы/Выходы - 54 (14 из которых могут работат также как выходы ШИМ)
- Аналоговые входы - 16
- Постоянный ток через вход/выход - 40 mA
- Постоянный ток для вывода - 3.3 В50 mA
- Флеш-память - 256 KB (из которых 8 KB используются для загрузчика)
- ОЗУ - 8 KB
- Энергонезависимая память - 4 KB

- Тактовая частота - 16 MHz

С учетом технических характеристик и экономических соображений для платформы ориентирования мы выбираем двигатель ZGA37RG DC Motor 12V 10RPM (Рисунок 17) в количестве двух штук. Для механизма параллельной структуры: ZGA37RG DC Motor 12V 60RPM.

Мотор-редуктор способен вращаться в обоих направлениях: по часовой и против часовой стрелки. Изменение направления вращения достигается путем изменения полярности напряжения на контактах двигателя[3].

Рисунок 17 – Двигатель постоянного тока с редуктором IG-32RGM-03TYPE-12V

Технические характеристики двигателя приведены в Таблице 1.

Таблица 1 – Технические характеристики двигателя IG-32RGM-03TYPE-12V

$P_{ном}$,Вт	$U_Я = U_В$,В	$I_Я$, А	$k_Р$	$\omega_{хх}$, об/мин	$\omega_н$, об/мин	$\omega_{вых}$, об/мин	$M_н$, кг·С/м
7	12	0,9	189	7300	5950	10	8.3

В таблице введены следующие обозначения:

- номинальная мощность двигателя $P_{ном}$;
- номинальное напряжение питания якоря $U_Я$;
- номинальное напряжение возбуждения $U_В$;
- частота вращения холостого хода $\omega_{хх}$;
- номинальная частота вращения двигателя $\omega_н$;
- номинальная частота вращения вала редуктора $\omega_{вых}$;
- номинальный ток якоря $I_Я$;Ф
- передаточное отношение $k_Р$;
- номинальный крутящий момент $M_н$.

Для считывания оборотов вала мотора используем абсолютный энкодер AS5600 (Рисунок 18).

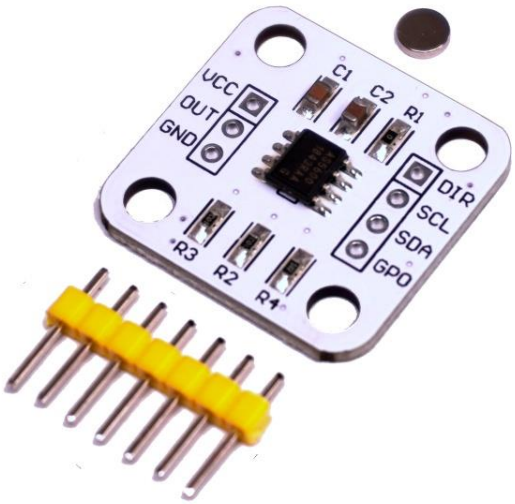

Рисунок 18 - AS5600

AS5600 - это компактный абсолютный энкодер с магнитным датчиком положения оси ротора. Этот уникальный датчик включает в себя четыре датчика Холла, позволяющих осуществлять измерение магнитного поля и преобразовывать его в напряжение. Устройство особенно полезно для определения угла, ось которого вращается.

Основой модуля является монолитная микросхема AS5600 в корпусе SOIC8. Адрес микросхемы на шине I2C задан по умолчанию (0x36) и его нельзя изменить, что накладывает определенные ограничения по количеству подключаемых к шине датчиков.

Модуль оснащен встроенным преобразователем напряжения.

Магнит, используемый с датчиком угла, должен обладать горизонтальной полярностью N/S. Расстояние между датчиком и магнитом может варьироваться от 0,5 до 3 мм зависит от мощности магнита.

С учетом 12-битного разрешения АЦП датчика и полного оборота магнита на 360°, можно получить 4096 шагов, что гарантирует точность датчика ориентировочно ~0,088° на 1 шаг поворота оси.

Технические характеристики:

- Микросхема: AS5600
- Интерфейс: I2C или ШИМ
- Диапазон углов: от 0 до 360°
- Минимальный шаг угла: 0,088°
- Максимальное измеряемое количество шагов за полный оборот: 4096
- Разрядность АЦП: 12 бит
- Диапазон температур: от -40° до 100°C
- Напряжение питания максимум: 6 В

Для управления двигателем используем драйвер TA6586 (Рисунок 19).

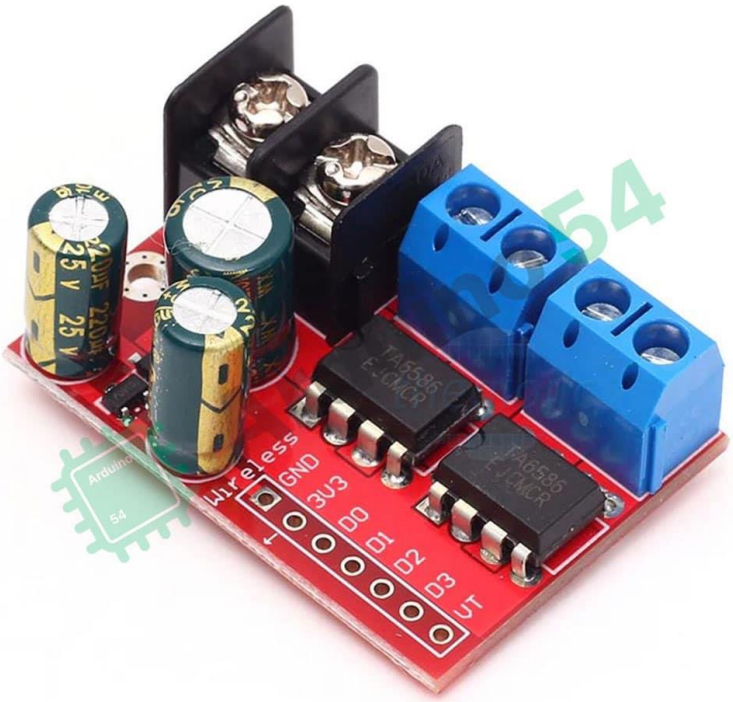

Рисунок 19 - TA6586

Технические характеристики:

- Модуль с двойным приводом 5А
- Модель: ZK-5AD
- Рабочее напряжение: DC 3,0 В-14 В
- Входное напряжение сигнала: DC 2,2 В-6,0 В
- Ток привода: 5А
- Постоянный ток: 10uA

В качестве датчика тока используем модуль ACS712 (Рисунок 20).

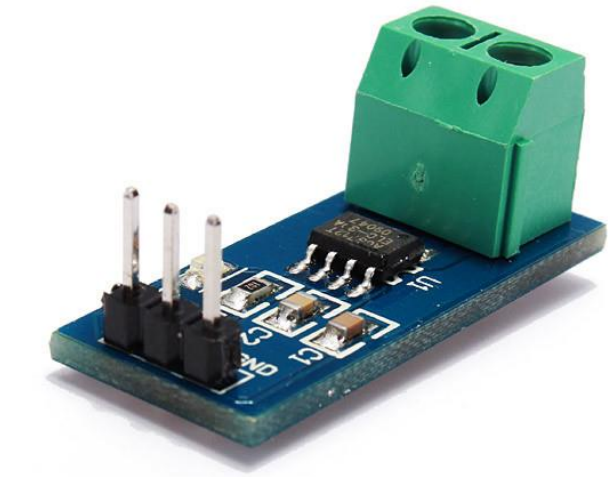

Рисунок 20 - ACS712

- Диапазон измерения тока - 0 - 30 А
- Чувствительность - 66 мВ/А
- Температурный диапазон - 40 .. +85 С
- Ток потребления не превышает - 11 мА
- Сопротивление внутреннего шунта - 1,2 мОм
- Гальваническая развязка, пробивное напряжение - 2,1 кВ

Блок питания (Рисунок 21) выберем из характеристик двигателя.

Рисунок 21 – блок питания

Технические характеристики:

- Вид питания - от сети 220В
- Напряжение адаптера питания - 12В
- Максимальная мощность, Вт - 100
- Выходное напряжение, В - 12
- Степень защиты - IP20
- Макс. выходной ток, А - 8.33

Для отображения измерений датчика тока выберем OLED 0.96" дисплей 128x64 (Рисунок 22).

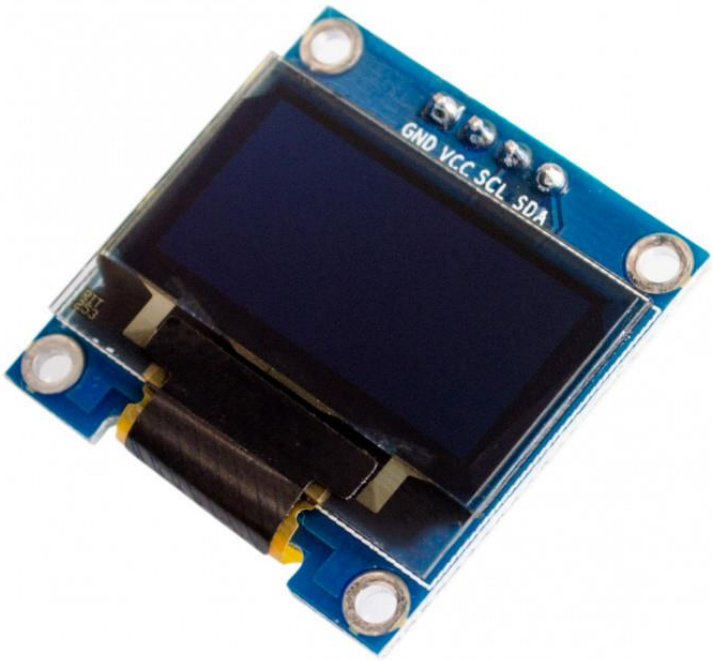

Рисунок 22 - OLED 0.96" дисплей 128x64

Технические характеристики:

- Тип матрицы: OLED
- Диагональ экрана: 0,96 дюйма
- Разрешение экрана: 128x64
- Цвет пикселя: белый
- Угол обзора: более 160 гр
- Коэффициент контрастности: 10000:1
- Яркость: >120 кд/м2
- Драйвер дисплея: SSD1306
- Интерфейс: I2C IIC
- Энергопотребление: до 40 мВт (при свечении всего экрана)
- Рабочее напряжение: 3 В - 5 В
- Размер платы: 29.3 x 27.6 мм

- Размер дисплея: 23.4 x 23.4 мм

5. Структурная схема системы управления

Рассмотрим принцип подключения двигателей и энкодеров к данным микроконтроллерам. Принципиальная схема подключения представлена на рисунке 23.

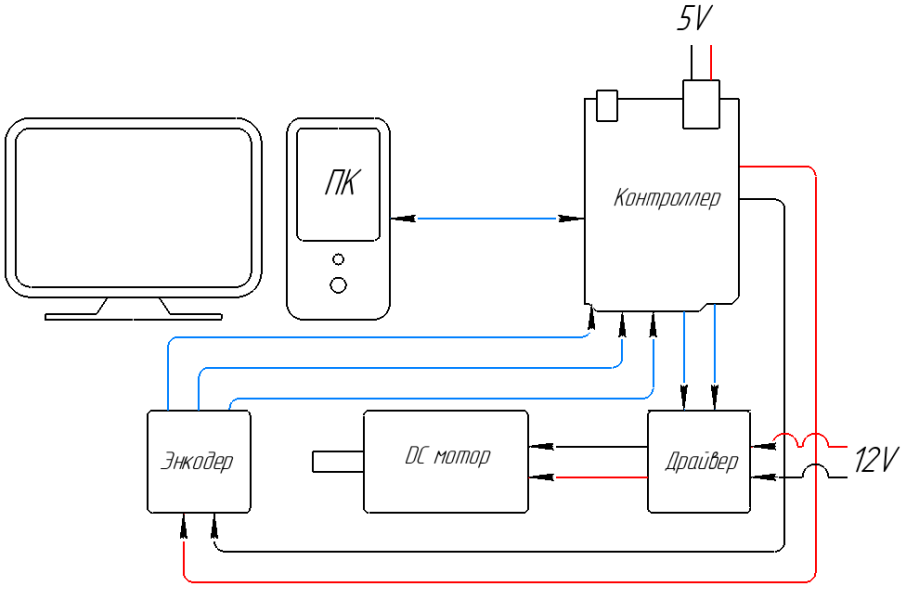

Рисунок 23 – Принципиальная схема подключения

1. Подключение DC мотора к драйверу:

- Подключите положительный (+) и отрицательный (-) выводы DC мотора к соответствующим выходам на драйвере.

- Подключите источник питания 12V к драйверу, чтобы обеспечить питание для DC мотора.

2. Подключение драйвера ZK5AD к микроконтроллеру:

- Соедините выходы управления драйвера (в случае ZK5AD это D0 и D1) с цифровыми пинами GPIO на микроконтроллере.

- Подключите общий заземляющий провод от микроконтроллера к драйверу, чтобы обеспечить общую точку заземления.

3. Подключение энкодера as5600 к микроконтроллеру:

- Подключите выводы датчика AS5600 к микроконтроллеру. Обычно датчик имеет следующие выводы:

- VDD: подключите к питанию 3.3V.

- GND: подключите к заземлению.

- OUT: подключите к цифровому пину GPIO.

4. Подключение микроконтроллера к компьютеру:

- Используйте USB-кабель для подключения микроконтроллера к компьютеру.

- Убедитесь, что на компьютере установлены необходимые драйвера (если требуется) для взаимодействия с микроконтроллером.

- Для программирования микроконтроллера и управления им с компьютера используйте соответствующее программное обеспечение.

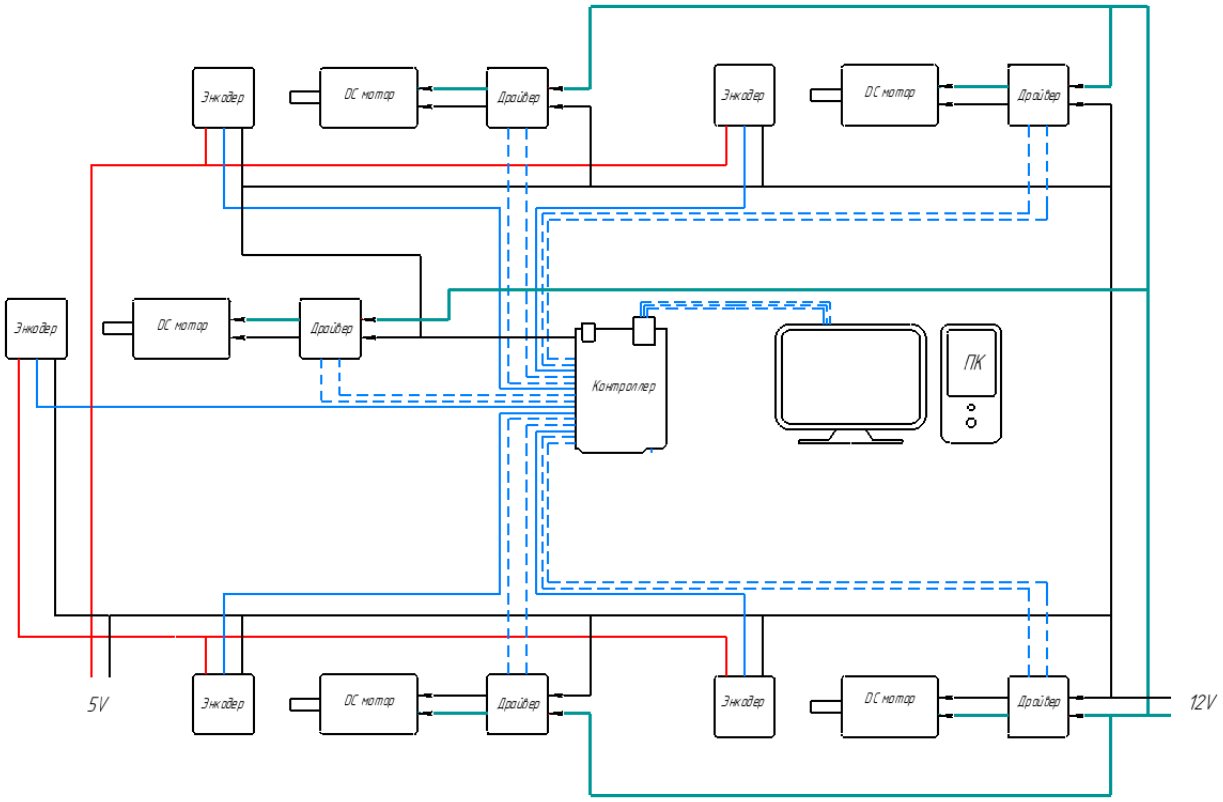

Рисунок 24 – Схема

Синей сплошной линией на схеме обозначены аналоговые каналы передачи данных, синим пунктиром – цифровые, красным – линия питания +5V, зелёным – + 12V, чёрным – GND.

Роль микроконтроллера в данной конфигурации выполняет Arduino MEGA, на ней производятся все вычислительные операции, связанные с PID-регулятором, а также контроль ШИМ и анализ и аппроксимация данных с энкодеров[2].

В схеме задействовано 5 аналоговых портов из 15 и 10 цифровых из 54 (см. Таблица 1).

Принципиальная схема управления изображена на рисунке 25.

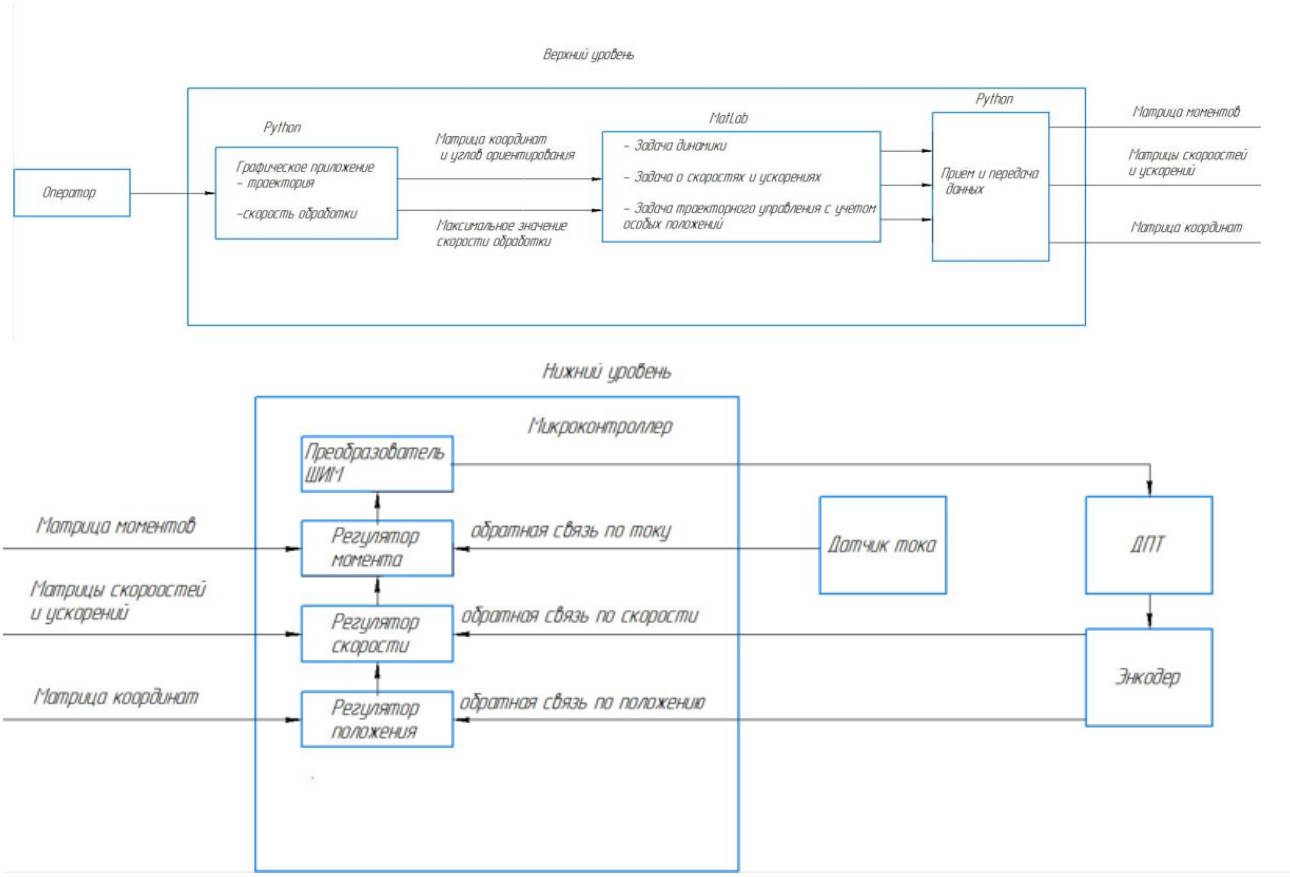

Рисунок 25 – Принципиальная схема управления

Условия и ограничения движения выходного звена механизма в пространстве задаются изначально пользователем, или, к примеру, программным обеспечением подготовки данных об обрабатываемой детали. Эти условия представляют собой набор координат точек на поверхности детали, через которых необходимо совершить движение рабочим органом на выходном звене механизма с определенной ориентацией относительно поверхности детали[3]. Все координаты направляются в верхний уровень системы управления для подготовки данных, необходимых для поворота двигателей. А именно: рассчитываются углы поворотов двигателей в сочленениях, их скорости, ускорения, момент, развиваемый двигателем при

движении. Все данные используются для планирования вращения каждого привода в двигателе для получения движения механизма, удовлетворяющего заданным условиям[1]. В зависимости от применяемой схемы управления приводами, на выходе с блока «Планирование траектории» получаются соответствующие наборы обобщенных параметров.

На нижнем уровне системы управления, задающие параметры сравниваются с текущими, и разница их (ошибка) поступает в алгоритм регулятора, который формирует управляющее воздействие таким образом, чтоб минимизировать ошибку. Управляющее воздействие от регулятора проходит схему усилителя, для того, чтоб преобразовать управляющий сигнал, в соответствующий сигнал для электромеханической системы (привода) соразмерный с величиной питающего сигнала для данного привода [9].

Текущее угловое положение привода, скорость вращения, а также развиваемый момент считываются и оцениваются датчиками, которые передают данный текущий набор параметров в модуль сравнения текущих параметров с задающими.

В режиме реального времени, с верхнего на нижний уровень приходят наборы задающих параметров, и система регуляторов, стремясь минимизировать ошибку, приводят приводы и их текущие параметры к задающим, управляя механизмом в целом, и удовлетворяя условиям, заданным пользователем[3].

6. Создание графического приложения

Основные компоненты и функции

Инициализация интерфейса

Программа начинается с инициализации основного окна приложения с использованием библиотеки Tkinter (рисунок 26)

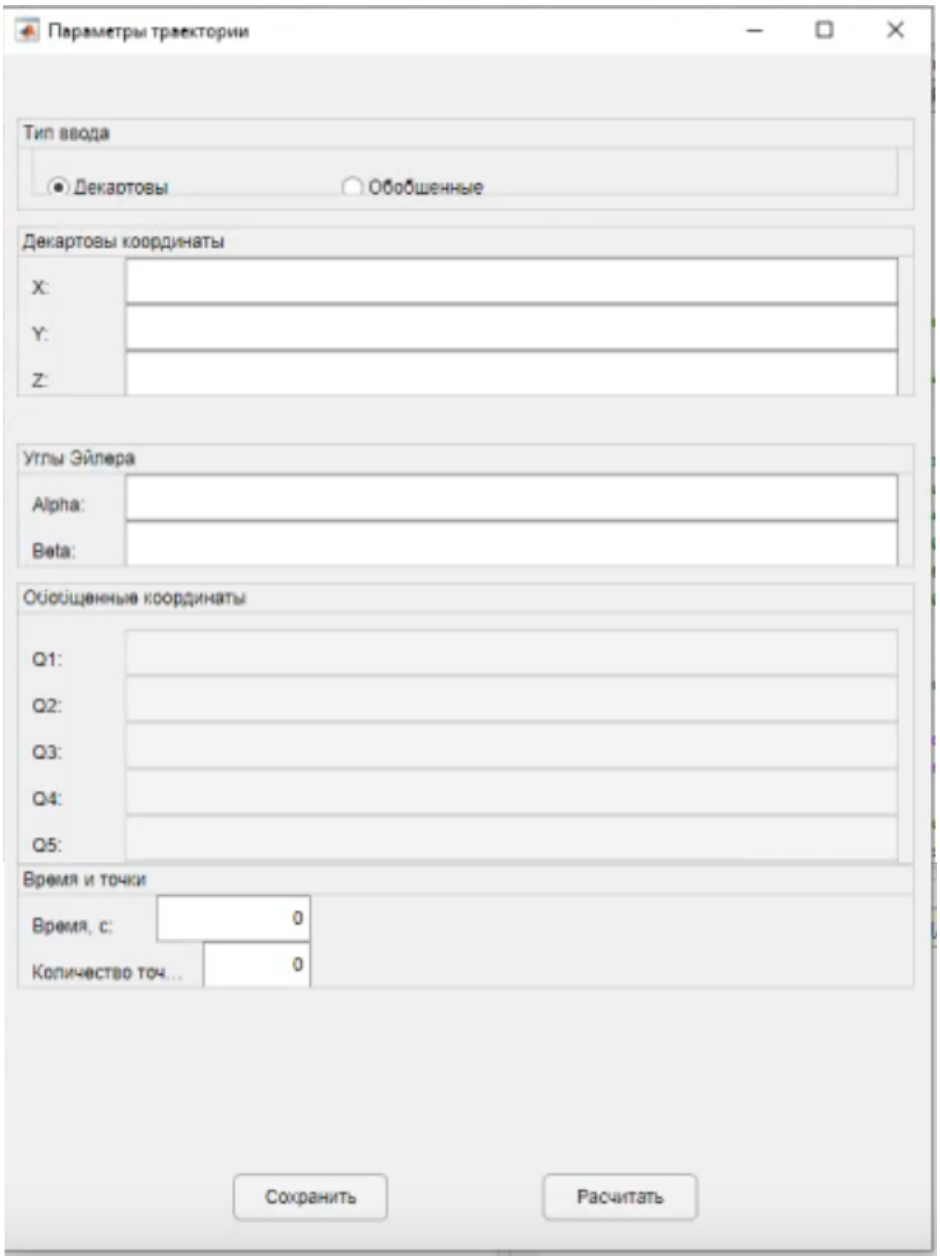

Рисунок 26 – Основное окно приложения

Основное окно содержит различные фреймы и виджеты для ввода данных:

- **Фрейм для выбора типа ввода**: позволяет пользователю выбирать между декартовыми координатами и обобщенными координатами.
- **Фрейм для декартовых координат**: содержит поля ввода для X, Y и Z координат, а также углов Эйлера (α, β, γ).
- **Фрейм для углов Эйлера**: содержит поля ввода для углов Эйлера (α, β).
- **Фрейм для обобщенных координат**: содержит поля ввода для обобщенных координат Q1-Q5.
- **Фрейм для ввода времени и количества точек**: позволяет пользователю вводить временные параметры и количество точек траектории.

Функция update_input_mode

Эта функция отвечает за обновление состояния полей ввода в зависимости от выбранного режима ввода. При выборе декартовых координат активируются соответствующие поля ввода, а поля для обобщенных координат становятся недоступными, и наоборот[3].

Функции создания полей ввода

Функции **create_coordinates_inputs**, **create_angles_inputs**, **create_generalized_coordinates_inputs** и **create_time_points_inputs** отвечают за создание и размещение полей ввода для различных типов данных:

- **Декартовы координаты**: X, Y, Z.
- **Углы Эйлера**: α (Alpha), β (Beta), γ (Gamma).
- **Обобщенные координаты**: Q1-Q5.
- **Временные параметры**: время и количество точек.

Функция submit

Эта функция выполняет проверку данных и их сохранение в файл. В зависимости от выбранного режима ввода данные сохраняются либо в файл **coordinates.txt** (для декартовых координат), либо в файл **generalized_coordinates.txt** (для обобщенных координат). Если существует файл, содержащий данные для другого режима ввода, он удаляется, чтобы избежать путаницы[3].

Функция выполняет следующие действия:

1. Получает данные из полей ввода.
2. Удаляет существующий файл с данными другого типа (если он существует).
3. Сохраняет введенные данные в соответствующий файл.
4. Выводит сообщение о успешном сохранении данных. Результат представлен на рисунке 27.

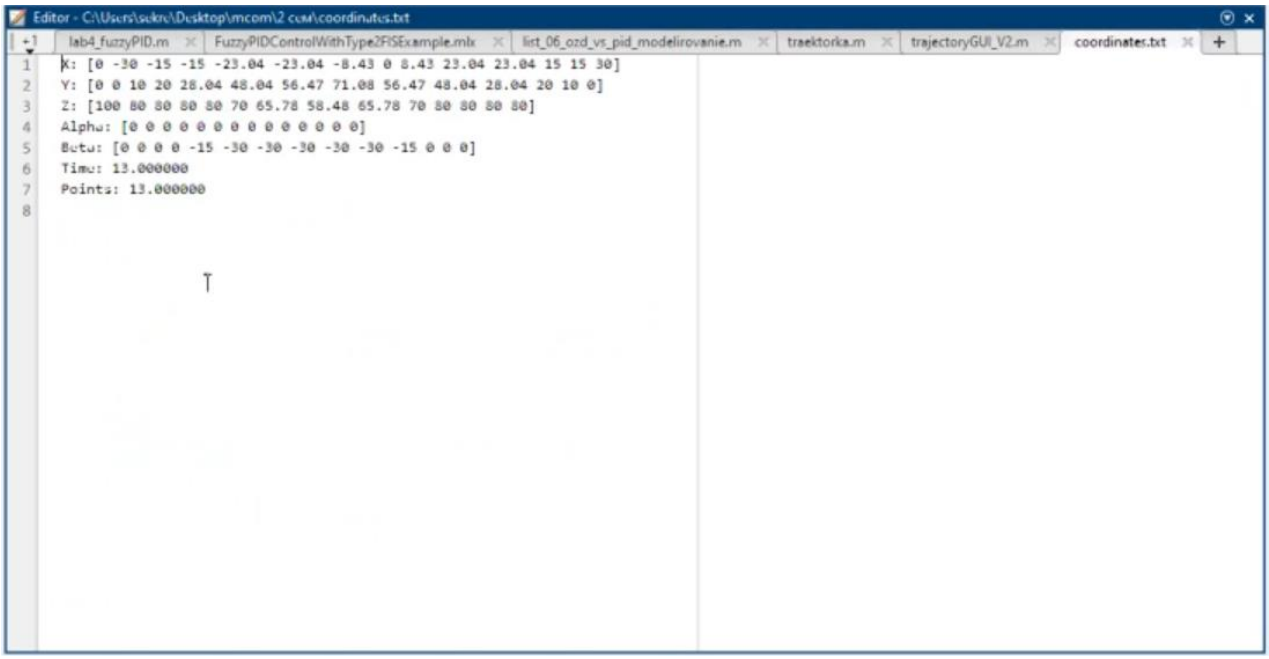

Рисунок 27 – Сохранение в файл

Созданный графический интерфейс позволяет удобно вводить данные для управления траекторией движения шестизвенного механизма. Четкая структура и использование фреймов обеспечивают интуитивно понятное расположение элементов интерфейса. Реализация функций для обновления режима ввода и сохранения данных гарантирует корректность и надежность работы программы.

Также программа выводит графики зависимости обобщённых координат от времени при одном и нескольких полиномах (рисунок 28).

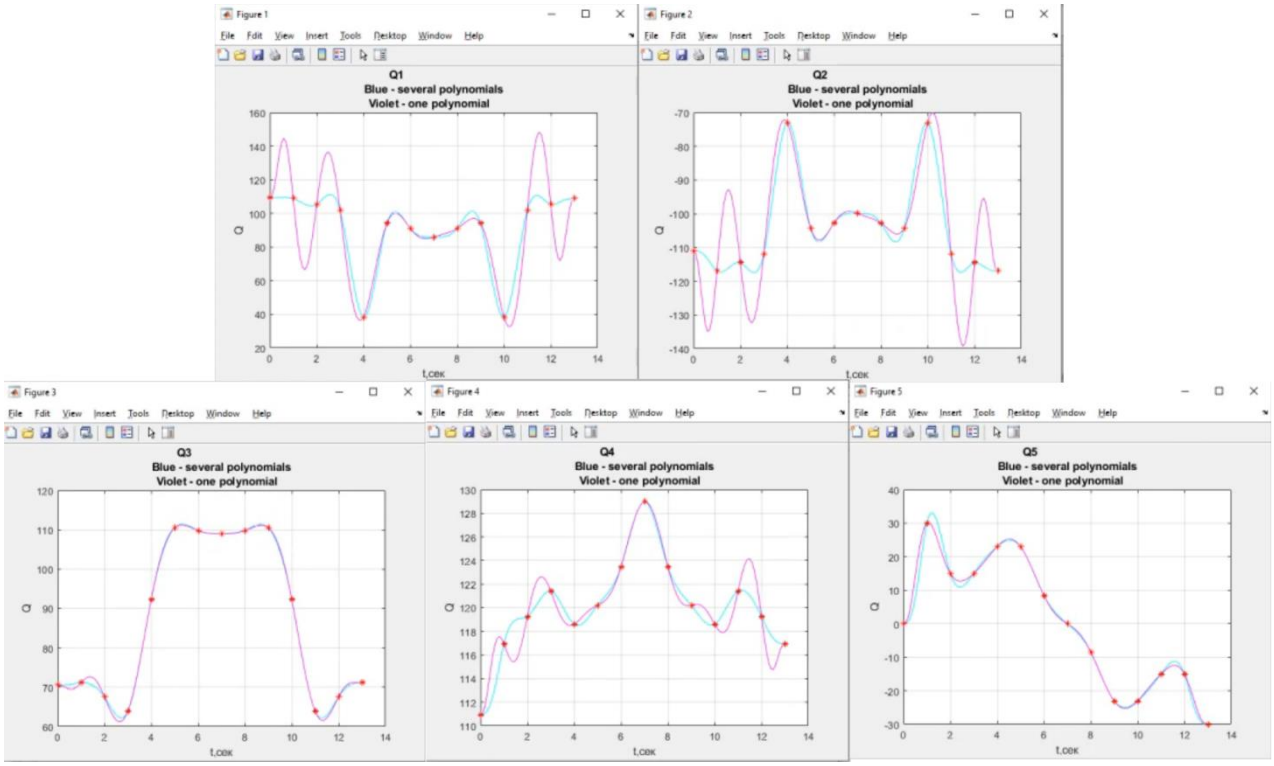

Рисунок 28 – Графики изменения обобщённых координат

Также выводятся прямые и обратные значения определителей (Рисунок 29).

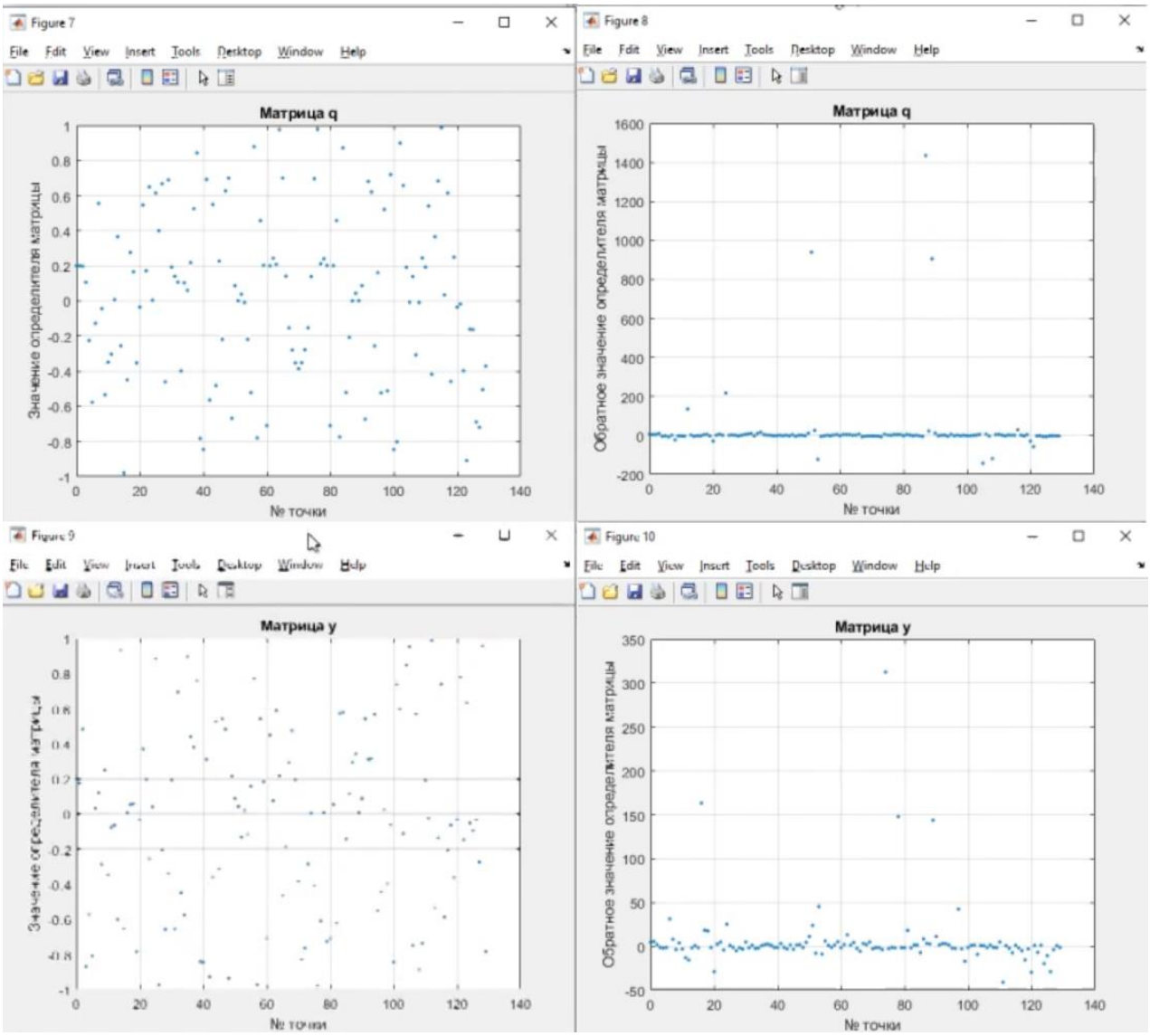

Рисунок 29 – Значения определителей матриц q и y

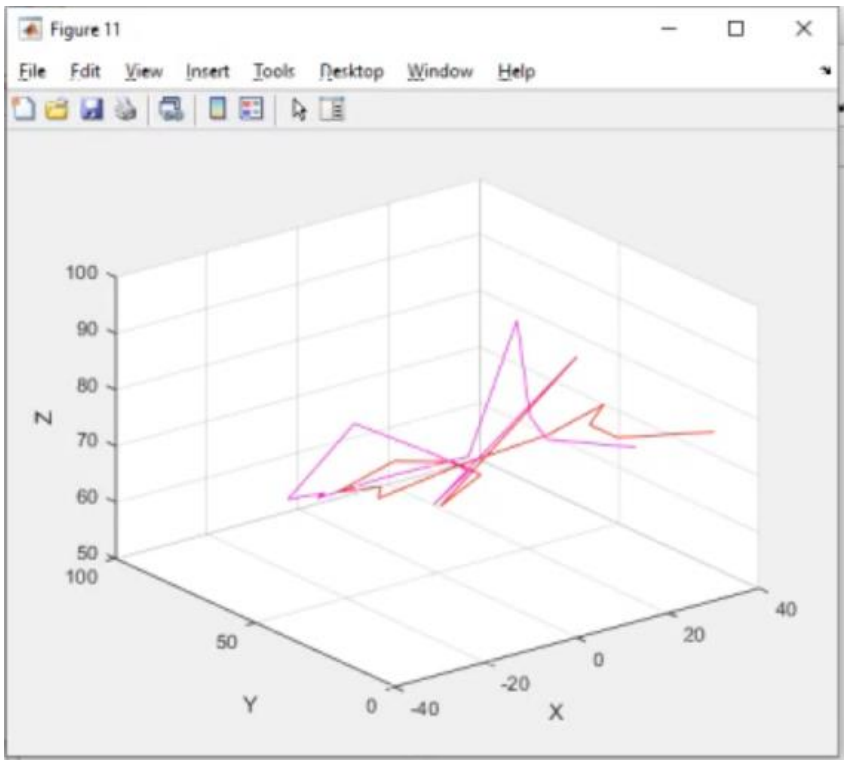

Рисунок 30 – Графики изменения обобщённых координат

Графическое представление ошибок (Рисунок 31)

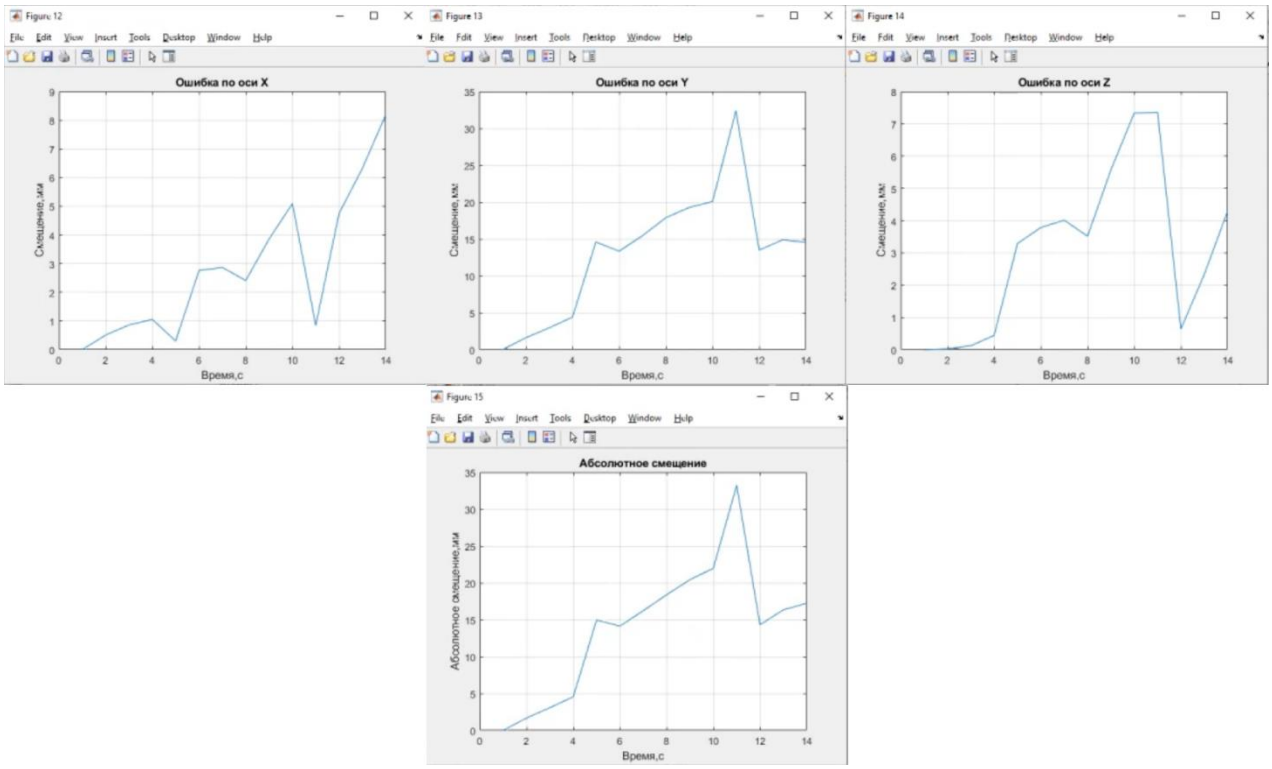

Рисунок 31 – Графики ошибки

Графики изменения обобщённых скоростей и ускорений (Рисунки 32-33)

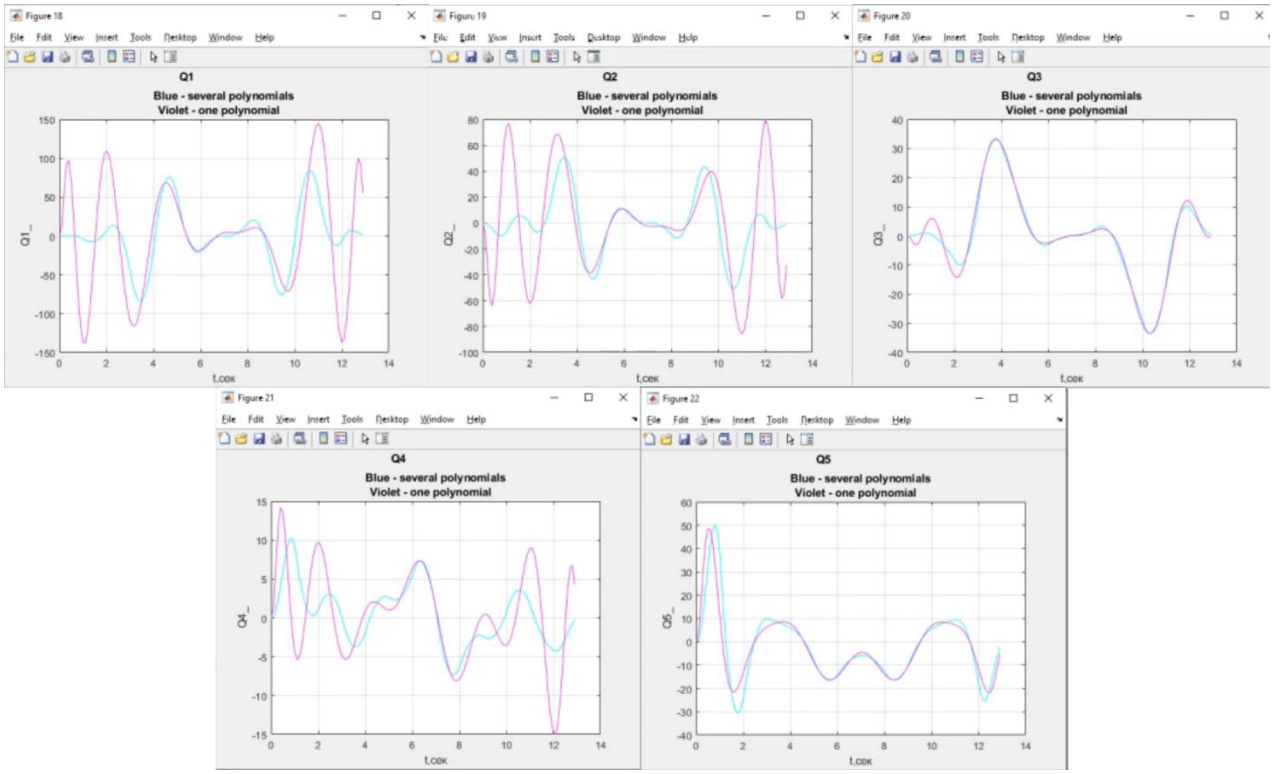

Рисунок 32 – Графики изменения обобщённых скоростей

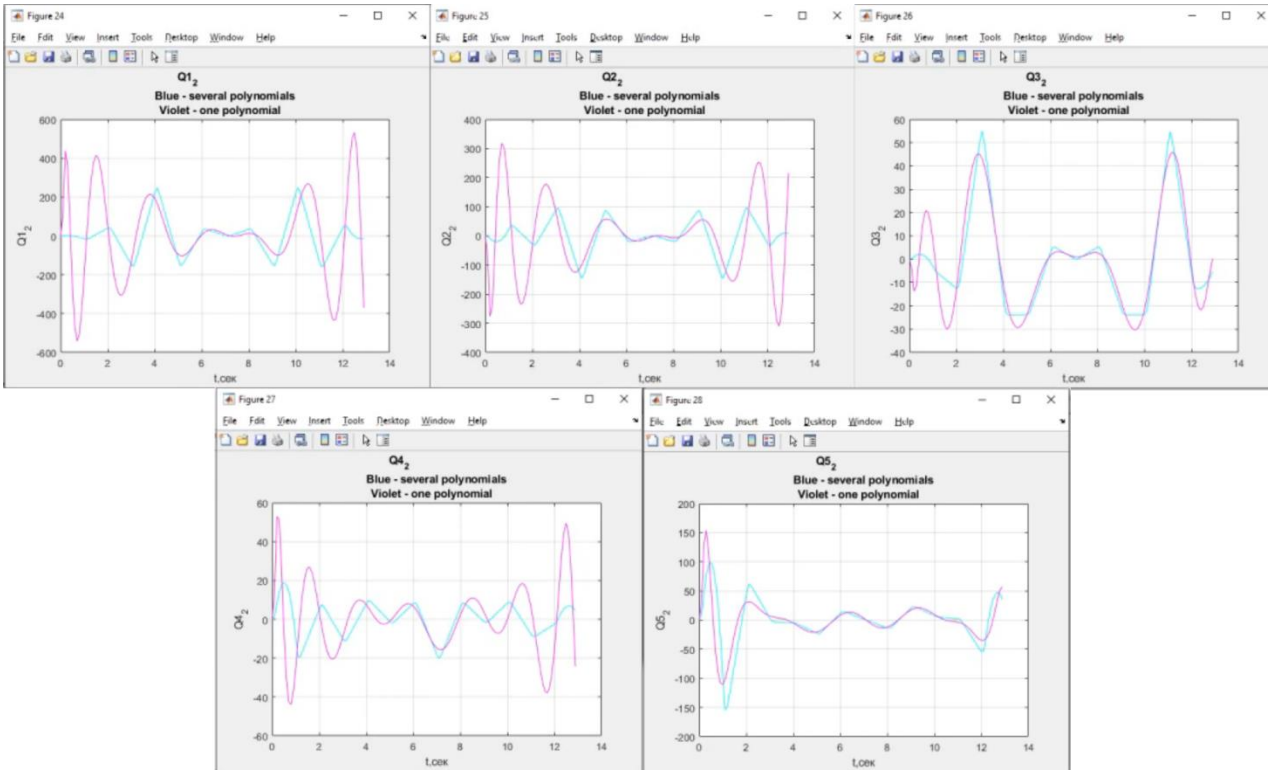

Рисунок 28 – Графики изменения обобщённых ускорений

Код программы приведен в **ПРИЛОЖЕНИИ А**.

Список использованных источников

1. Пащенко В.Н., Гайгеров М.А., Зайков С.А., Просвернин А.А., Романов И.Ю., Скадин А.В. Методическое пособие. МГТУ, 2025. С. 33-47
2. Пащенко В. Н. Глава 12. Разработка роботов параллельной структуры для совместного относительного манипулирования технологическими системами : монография / В.Н. Пащенко - Новые механизмы в современной робототехнике. Москва, 2018. С. 268-294.
3. Пащенко В.Н., Власовский А.И., Мясоедов К., Вейсман П.И. Параметрический синтез механизмов совместного относительного манипулирования // Сборник трудов конференции МИКМУС-2018. М.: ИМАШ РАН, 2019. С. 490-493
4. Пащенко В.Н. Decision of the direct position problem of the joint Relative manipulation mechanism with five degrees of Freedom : научная работа / В. Н. Пащенко, Д.И. Мелещенко, Д.И. Малышев, В.С. Кузьмина. - Int. J. Of Applied Mechanics and Engineering, 2018, vol.23, No.4, pp.1025-1033 DOI: 10.2478/ijame-2018-0059.
5. Машков, К. Ю. Состав и характеристики мобильных роботов : учебное пособие / К. Ю. Машков, В. И. Рубцов, И. В. Рубцов. — Москва : МГТУ им. Н.Э. Баумана, 2014. — 75 с. — ISBN 978-5-7038-3866-2. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/58390
6. Предко, М. Устройства управления роботами : учебное пособие / М. Предко. — Москва : ДМК Пресс, 2010. — 404 с. — ISBN 5-94074-226-2. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/40006
7. Лукинов, А. П. Проектирование мехатронных и робототехнических устройств : учебное пособие / А. П. Лукинов. — Санкт-Петербург : Лань, 2021. — 608 с. — ISBN 978-5-8114-1166-5. — Текст : электронный // Лань

: электронно-библиотечная система. — URL: https://e.lanbook.com/book/168366

8. Ощепков, А. Ю. Системы автоматического управления: теория, применение, моделирование в MATLAB : учебное пособие для вузов / А. Ю. Ощепков. — 4-е изд., стер. — Санкт-Петербург : Лань, 2021. — 208 с. — ISBN 978-5-8114-8544-4. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/177027

ПРИЛОЖЕНИЕ А

```
import tkinter as tk
from tkinter import ttk
import os


class TrajectoryGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Параметры траектории")

        # Фрейм для выбора ввода координат или обобщенных координат
        mode_frame = ttk.LabelFrame(root, text="Тип ввода")
        mode_frame.grid(row=0, column=0, padx=10, pady=10, sticky="ew")

        self.input_mode = tk.StringVar(value="coordinates")

        coordinates_radio = ttk.Radiobutton(mode_frame, text="Декартовы ",
variable=self.input_mode,
                                            value="coordinates",
command=self.update_input_mode)
        coordinates_radio.grid(row=0, column=0, padx=5, pady=5)

        generalized_radio = ttk.Radiobutton(mode_frame, text="Обобщенные",
variable=self.input_mode,
                                            value="generalized",
command=self.update_input_mode)
        generalized_radio.grid(row=0, column=1, padx=5, pady=5)

        coord_frame = ttk.LabelFrame(root, text="Декартовы координаты")
        coord_frame.grid(row=1, column=0, padx=10, pady=10, sticky="ew")

        self.create_coordinates_inputs(coord_frame)

        angle_frame = ttk.LabelFrame(root, text="Углы Эйлера")
        angle_frame.grid(row=2, column=0, padx=10, pady=10, sticky="ew")

        self.create_angles_inputs(angle_frame)

        q_frame = ttk.LabelFrame(root, text="Обощенные координаты")
        q_frame.grid(row=3, column=0, padx=10, pady=10, sticky="ew")

        self.create_generalized_coordinates_inputs(q_frame)

        time_points_frame = ttk.LabelFrame(root, text="Время и точки")
        time_points_frame.grid(row=4, column=0, padx=10, pady=10, sticky="ew")

        self.create_time_points_inputs(time_points_frame)

        self.submit_button = ttk.Button(root, text="Сохранить",
command=self.submit)
        self.submit_button.grid(row=5, column=0, padx=10, pady=10)

        self.update_input_mode()

    def create_coordinates_inputs(self, frame):
        ttk.Label(frame, text="X:").grid(row=0, column=0, padx=5, pady=5,
sticky="e")
        self.x_entry = ttk.Entry(frame)
        self.x_entry.grid(row=0, column=1, padx=5, pady=5)
```

```
        ttk.Label(frame, text="Y:").grid(row=1, column=0, padx=5, pady=5,
sticky="e")
        self.y_entry = ttk.Entry(frame)
        self.y_entry.grid(row=1, column=1, padx=5, pady=5)

        ttk.Label(frame, text="Z:").grid(row=2, column=0, padx=5, pady=5,
sticky="e")
        self.z_entry = ttk.Entry(frame)
        self.z_entry.grid(row=2, column=1, padx=5, pady=5)

    def create_angles_inputs(self, frame):
        ttk.Label(frame, text="Alpha:").grid(row=0, column=0, padx=5, pady=5,
sticky="e")
        self.alpha_entry = ttk.Entry(frame)
        self.alpha_entry.grid(row=0, column=1, padx=5, pady=5)

        ttk.Label(frame, text="Beta:").grid(row=1, column=0, padx=5, pady=5,
sticky="e")
        self.beta_entry = ttk.Entry(frame)
        self.beta_entry.grid(row=1, column=1, padx=5, pady=5)

        ttk.Label(frame, text="Gamma:").grid(row=2, column=0, padx=5, pady=5,
sticky="e")
        self.gamma_entry = ttk.Entry(frame)
        self.gamma_entry.grid(row=2, column=1, padx=5, pady=5)

    def create_generalized_coordinates_inputs(self, frame):
        ttk.Label(frame, text="Q1:").grid(row=0, column=0, padx=5, pady=5,
sticky="e")
        self.q1_entry = ttk.Entry(frame, state='disabled')
        self.q1_entry.grid(row=0, column=1, padx=5, pady=5)

        ttk.Label(frame, text="Q2:").grid(row=1, column=0, padx=5, pady=5,
sticky="e")
        self.q2_entry = ttk.Entry(frame, state='disabled')
        self.q2_entry.grid(row=1, column=1, padx=5, pady=5)

        ttk.Label(frame, text="Q3:").grid(row=2, column=0, padx=5, pady=5,
sticky="e")
        self.q3_entry = ttk.Entry(frame, state='disabled')
        self.q3_entry.grid(row=2, column=1, padx=5, pady=5)

        ttk.Label(frame, text="Q4:").grid(row=3, column=0, padx=5, pady=5,
sticky="e")
        self.q4_entry = ttk.Entry(frame, state='disabled')
        self.q4_entry.grid(row=3, column=1, padx=5, pady=5)

        ttk.Label(frame, text="Q5:").grid(row=4, column=0, padx=5, pady=5,
sticky="e")
        self.q5_entry = ttk.Entry(frame, state='disabled')
        self.q5_entry.grid(row=4, column=1, padx=5, pady=5)

        ttk.Label(frame, text="Q6:").grid(row=5, column=0, padx=5, pady=5,
sticky="e")
        self.q6_entry = ttk.Entry(frame, state='disabled')
        self.q6_entry.grid(row=5, column=1, padx=5, pady=5)

    def create_time_points_inputs(self, frame):
        ttk.Label(frame, text="Время, с:").grid(row=0, column=0, padx=5,
pady=5, sticky="e")
        self.time_entry = ttk.Entry(frame)
        self.time_entry.grid(row=0, column=1, padx=5, pady=5)
```

```
        ttk.Label(frame, text="Количество точек:").grid(row=1, column=0,
padx=5, pady=5, sticky="e")
        self.points_entry = ttk.Entry(frame)
        self.points_entry.grid(row=1, column=1, padx=5, pady=5)

    def update_input_mode(self):
        if self.input_mode.get() == "coordinates":
            self.enable_coordinates(True)
            self.enable_generalized_coordinates(False)
        else:
            self.enable_coordinates(False)
            self.enable_generalized_coordinates(True)

    def enable_coordinates(self, enable):
        state = 'normal' if enable else 'disabled'
        self.x_entry.config(state=state)
        self.y_entry.config(state=state)
        self.z_entry.config(state=state)
        self.alpha_entry.config(state=state)
        self.beta_entry.config(state=state)
        self.gamma_entry.config(state=state)

    def enable_generalized_coordinates(self, enable):
        state = 'normal' if enable else 'disabled'
        self.q1_entry.config(state=state)
        self.q2_entry.config(state=state)
        self.q3_entry.config(state=state)
        self.q4_entry.config(state=state)
        self.q5_entry.config(state=state)
        self.q6_entry.config(state=state)

    def submit(self):
        time = self.time_entry.get()
        points = self.points_entry.get()

        if self.input_mode.get() == "coordinates":
            x = self.x_entry.get()
            y = self.y_entry.get()
            z = self.z_entry.get()
            alpha = self.alpha_entry.get()
            beta = self.beta_entry.get()
            gamma = self.gamma_entry.get()

            # Удаление файла с обобщенными координатами, если он существует
            if os.path.exists('generalized_coordinates.txt'):
                os.remove('generalized_coordinates.txt')

            # Сохранение координат в файл
            with open('coordinates.txt', 'w') as f:
                f.write(
                    f"X: {x}\nY: {y}\nZ: {z}\nAlpha: {alpha}\nBeta:
{beta}\nGamma: {gamma}\nTime: {time}\nPoints: {points}\n")
        else:
            q1 = self.q1_entry.get()
            q2 = self.q2_entry.get()
            q3 = self.q3_entry.get()
            q4 = self.q4_entry.get()
            q5 = self.q5_entry.get()
            q6 = self.q6_entry.get()

            # Удаление файла с координатами, если он существует
            if os.path.exists('coordinates.txt'):
                os.remove('coordinates.txt')
```

```
            # Сохранение обобщенных координат в файл
            with open('generalized_coordinates.txt', 'w') as f:
                f.write(f"Q1: {q1}\nQ2: {q2}\nQ3: {q3}\nQ4: {q4}\nQ5: {q5}\nQ6:
{q6}\nTime: {time}\nPoints: {points}\n")

        print("Данные успешно сохранены.")


# Создание основного окна приложения
root = tk.Tk()
app = TrajectoryGUI(root)
root.mainloop()
```